

EcoTrack – Personal Carbon AI Footprint Tracker

Project Description:

EcoTrack is an AI-powered web application designed to help individuals monitor, analyze, and reduce their daily carbon footprint through intelligent activity tracking and personalized sustainability recommendations. The platform enables users to record their daily transportation, food consumption, and household energy usage, automatically calculate carbon emissions, visualize environmental impact through interactive dashboards, and receive AI-generated suggestions for adopting more sustainable habits.

The application consists of several integrated modules that work together to provide a seamless user experience. The **Authentication Module** allows users to securely register and log in using Firebase Authentication. The **Habit Logger Module** enables users to record daily transportation, dietary habits, and household energy consumption. Based on the information entered, the **Carbon Footprint Calculator** estimates carbon emissions using predefined emission factors for different activities.

To help users monitor their progress, the **Dashboard Module** presents key environmental metrics such as total carbon emissions, Eco Score, carbon budget utilization, emission trends, and category-wise emission distribution through interactive charts and visual indicators. The **History Module** stores all previous activity records in Firebase Cloud Firestore, allowing users to review, analyze, and compare their environmental impact over time.

EcoTrack also integrates **Groq's LLaMA 3.3-70B Versatile** large language model to generate personalized eco-friendly recommendations. The **AI Insights Module** analyzes the user's recorded activities and provides practical suggestions to reduce carbon emissions, encouraging environmentally responsible decision-making and sustainable lifestyle changes.

The application is developed using **React.js** with **Vite** for building a fast, responsive, and interactive frontend, while the backend is implemented using the **Flask** framework to handle carbon emission calculations and API requests. **Firebase Authentication** ensures secure user authentication, and **Cloud Firestore** provides scalable cloud-based storage for user profiles and activity logs. The application is deployed using **Vercel** for the frontend and **Render** for the backend, ensuring reliable cloud hosting and accessibility across multiple devices.

By combining cloud computing, artificial intelligence, interactive data visualization, and carbon footprint analytics, EcoTrack provides users with an intelligent platform for understanding their environmental impact and promoting sustainable living. The project demonstrates the practical application of modern web technologies and AI in addressing real-world environmental challenges while encouraging individuals to make informed, eco-conscious decisions in their daily lives.

Scenarios

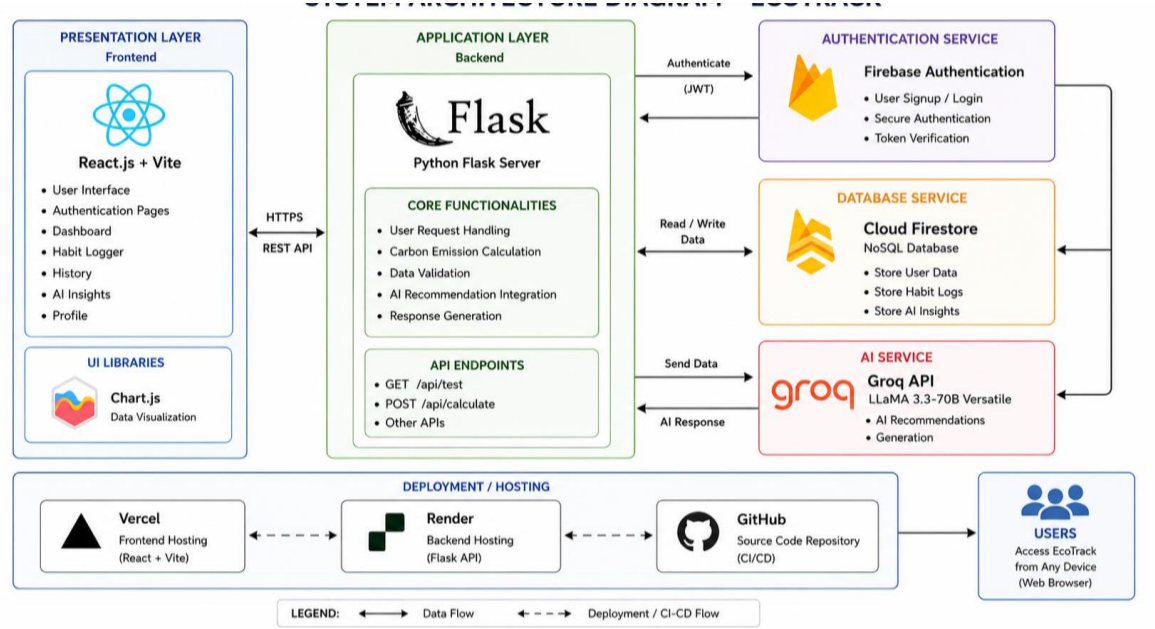
Scenario 1: A user logs their daily travel activity by selecting a transport mode (e.g., car petrol), entering distance travelled in kilometres, and specifying the number of passengers for carpooling. The system calculates travel emissions using science-backed coefficients, saves the record to Firebase Firestore, and displays the result on the dashboard gauge alongside a comparison against the user’s daily carbon budget.

Scenario 2: A user logs their food consumption by selecting a diet type (meat-heavy, omnivore, vegetarian, or vegan) and indicating whether food was wasted. The system applies a 10% emission penalty for food waste, computes the food carbon score, and adds it to the daily total displayed on the dashboard with a colour-coded progress indicator.

Scenario 3: After logging travel, food, and energy habits, the user navigates to the Suggestions page. The system submits the daily breakdown to the Groq API using the llama-3.3-70b-versatile model, which generates three personalised and actionable eco-friendly lifestyle swap recommendations with estimated CO₂ savings displayed as styled suggestion cards.

Scenario 4: The user opens the History page to review their carbon footprint trends. The dashboard renders a 30-day line chart with a budget reference line, a 14-day stacked bar breakdown by category, and a colour-coded heatmap calendar. Summary statistics including 7-day average, best day, worst day, and monthly total are displayed above the charts.

Technical Architecture:



Description: The System Architecture Diagram illustrates the overall design and workflow of the EcoTrack application. It demonstrates how different software components communicate to deliver a secure, scalable, and user-friendly carbon footprint tracking system.

The architecture follows a three-tier client-server model consisting of the Presentation Layer (React.js + Vite), Application Layer (Flask Backend), and Service Layer (Firebase Authentication, Cloud Firestore, and Groq AI API). Users interact with the frontend through a responsive web interface, where requests are sent securely to the Flask REST API. The backend processes business logic, calculates carbon emissions, validates user input, stores and retrieves data from Cloud Firestore, and communicates with the AI service to generate personalized sustainability recommendations. Authentication is handled by Firebase Authentication, ensuring secure user login and registration, while Cloud Firestore stores user profiles, activity logs, and emission records. The frontend is deployed on Vercel, the backend API is hosted on Render, and the complete source code is maintained on GitHub, enabling version control and continuous deployment.

This architecture provides modularity, scalability, security, and maintainability, allowing each component to function independently while ensuring seamless communication across the entire EcoTrack ecosystem.

Pre-requisites:

- **React.js Framework Knowledge:** <https://react.dev/learn>
- **Flask Framework Knowledge:** <https://flask.palletsprojects.com>
- **Groq API Familiarity:** <https://console.groq.com/docs/openai>
- **Firebase Setup:** <https://firebase.google.com/docs>
- **Carbon Interface API:** <https://docs.carboninterface.com>
- **HTML, CSS, and JavaScript Skills:** <https://www.w3schools.com>
- **Python Programming Proficiency:** <https://docs.python.org/3/>
- **Tailwind CSS Fundamentals:** <https://tailwindcss.com/docs>
- **Version Control with Git:** <https://git-scm.com/docs/git>

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Groq for social media content generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask and the Groq API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Caption Generator, Hashtag Suggester, and Call-to-Action Generator.
- **Activity 2.2:** Implement the Flask backend to manage routing and user input processing, ensuring smooth API interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates with Flask's render_template to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 1: Requirement Analysis and System Architecture

The first milestone of the EcoTrack project focused on identifying the problem, selecting suitable technologies, designing the overall system architecture, and preparing the development environment. This phase established a strong foundation for the implementation of the application by defining the project objectives, system workflow, technology stack, and software architecture. Careful planning during this stage ensured that the application would be scalable, secure, and capable of providing accurate carbon footprint tracking along with AI-powered sustainability recommendations.

Activity 1.1: Requirement Gathering and Problem Identification

Before beginning the development of EcoTrack, it was essential to understand the existing challenges associated with personal carbon footprint tracking. Many individuals are unaware of the environmental impact of their daily transportation, food consumption, and energy usage. Existing solutions often lack personalization, visual analytics, and intelligent recommendations that encourage users to adopt sustainable habits.

The following activities were performed during the requirement analysis phase:

- Study existing carbon footprint tracking applications and sustainability platforms.
- Identify limitations of current systems, including the lack of AI-driven recommendations and interactive visualization.
- Define the primary objectives of the EcoTrack application.
- Identify target users such as students, professionals, and environmentally conscious individuals.
- Gather functional and non-functional requirements for the proposed system.

The outcome of this activity was a clear understanding of the project requirements and the definition of the application's overall scope.

Activity 1.2: Technology Stack Selection

- Selecting the appropriate technologies was an important step in ensuring the successful development of EcoTrack. The chosen technologies provide a modern, scalable, and secure environment for both application development and deployment.
- The selected technology stack includes:

Component	Technology
Frontend	React.js with Vite
Backend	Flask (Python)
Authenticati on	Firebase Authentication
Database	Firebase Cloud Firestore
AI Model	Groq LLaMA 3.3-70B Versatile
Charts	Chart.js
Deployment	Vercel & Render
Version Control	Git & GitHub

- The combination of these technologies enables efficient frontend development, secure authentication, cloud-based data storage, REST API communication, AI integration, and scalable deployment.

Activity 1.3: Define the System Architecture

A modular three-tier architecture was designed to ensure maintainability, scalability, and efficient communication between different components of the application.

The system architecture consists of:

- Presentation Layer developed using React.js and Vite.
- Application Layer developed using Flask REST APIs.
- Authentication Layer using Firebase Authentication.
- Database Layer using Cloud Firestore.
- Artificial Intelligence Layer using Groq's LLaMA 3.3-70B model.
- Cloud Deployment using Render and Vercel.

The architecture enables secure communication between users, backend services, cloud storage, and AI components while maintaining a responsive and user-friendly interface.

Activity 1.4: Set Up the Development Environment

The development environment was configured to support both frontend and backend development while ensuring smooth integration between all project components.

The following setup activities were completed:

- Install Python 3.12 and required Python packages.
- Install Node.js and npm for frontend development.
- Create a React project using Vite.
- Configure the Flask backend application.
- Set up Firebase Authentication and Cloud Firestore.
- Install required frontend and backend dependencies.
- Configure Git and GitHub for version control.
- Create environment variable files (.env) for secure API key management.
- Connect the frontend with backend REST APIs.
- Verify the complete development environment before implementation.

The required dependencies were installed using the following commands:

- **Backend:**

```
python -m venv venv  
pip install -r requirements.txt  
python app.py
```

- **Frontend:**

```
npm install
```

```
npm run dev
```

The project structure was then organized into separate frontend and backend directories, allowing independent development and easier maintenance throughout the project lifecycle.

End of Milestone 1

This milestone established the technical foundation of the EcoTrack application by identifying project requirements, selecting appropriate technologies, designing the overall system architecture, and configuring the complete development environment. The successful completion of this milestone enabled the implementation of the application's core functionalities in the subsequent phases.

Milestone 2: Backend Development

In this milestone, the primary focus was on developing the backend of the EcoTrack application using the **Flask** framework. The backend is responsible for processing user requests, validating input data, calculating carbon emissions, and communicating with the frontend through RESTful APIs. It also acts as the central component that integrates Firebase services and the AI recommendation module.

The backend was designed to provide accurate carbon emission calculations based on user activities related to transportation, food consumption, and household energy usage. Additionally, it ensures efficient data processing and secure communication between different modules of the application.

Activity 2.1: Develop the Carbon Emission Calculation Module

The Carbon Emission Calculation Module is the core component of the EcoTrack backend. It calculates the total carbon emissions generated by a user's daily activities using predefined emission factors. The module processes transportation, dietary habits, and household energy consumption separately before combining them into a single emission value.

The following activities were carried out during the development of this module:

- Define emission factors for different transportation modes.
- Implement emission calculations for food consumption.
- Calculate household energy emissions based on energy usage.
- Validate user inputs before performing calculations.
- Calculate the total carbon footprint by combining emissions from all categories.
- Return the calculated emission values in JSON format for frontend visualization.

The emission calculation process is divided into three major categories:

Transportation Emission Calculation

The transportation module calculates carbon emissions based on the selected mode of transport, travel distance, and number of passengers. Different emission factors are assigned to vehicles such as petrol cars, diesel cars, buses, trains, flights, bicycles, and walking.

Food Emission Calculation

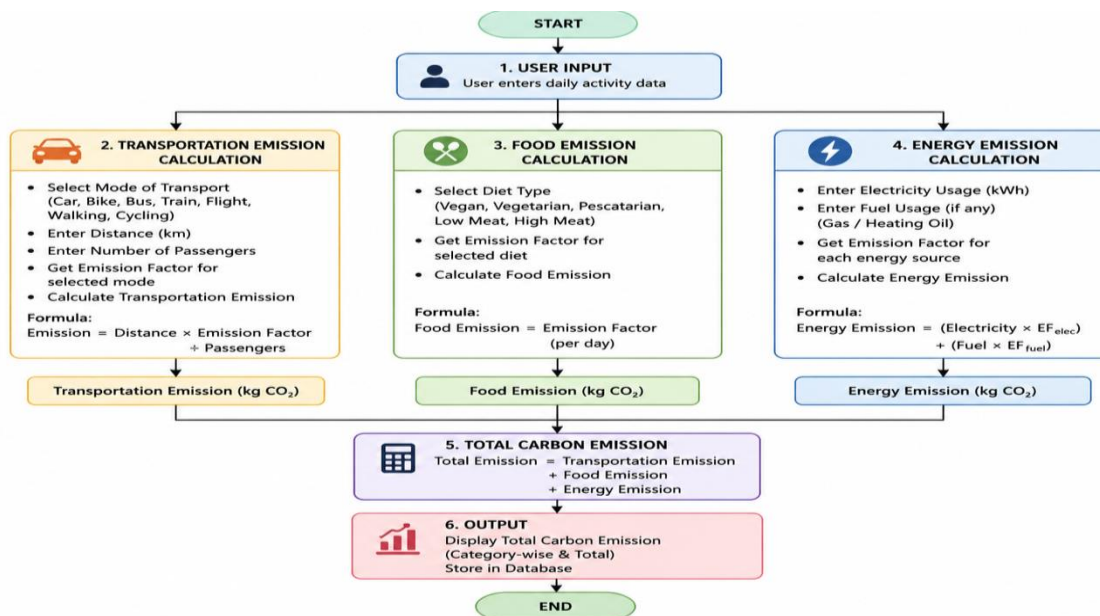
The food module estimates carbon emissions according to the user's dietary preferences, including vegan, vegetarian, pescatarian, low-meat, and high-meat diets. Each diet type has a predefined emission factor that contributes to the overall carbon footprint.

Energy Consumption Calculation

The energy module calculates emissions generated through household electricity and fuel consumption. The calculation considers the amount of energy consumed and distributes the emissions among household members where applicable.

Total Carbon Emission

The emissions from transportation, food, and energy usage are combined to generate the user's total daily carbon footprint. The calculated values are returned to the frontend and displayed through the Dashboard and History modules.



Activity 2.2: Flask REST API Development

To facilitate communication between the frontend and backend, RESTful APIs were developed using the Flask framework. These APIs receive requests from the React frontend, process the submitted data, perform carbon emission calculations, and return the results in JSON format.

The backend also validates user input and ensures reliable communication between different modules of the application.

The following APIs were implemented:

API 1 – Backend Status API

Endpoint

GET /api/test

Purpose

Returns the current status of the backend server to verify that the Flask application is running successfully.

API 2 – Carbon Emission Calculation API

Endpoint

POST /api/calculate

Purpose

Receives user activity data, calculates carbon emissions, and returns category-wise and total emission values.

Backend Functionalities

The Flask backend performs the following operations:

- Receives user requests from the frontend.
- Validates transportation, food, and energy inputs.
- Calculates carbon emissions using predefined emission factors.
- Generates JSON responses for frontend visualization.
- Handles exceptions and invalid requests gracefully.
- Supports integration with Firebase and AI recommendation services.

Backend Testing

The developed APIs were tested using both the frontend application and manual API requests to ensure correct calculations and proper response formatting. Successful testing confirmed that the backend processed user inputs accurately and communicated effectively with the frontend.

End of Milestone 2

The successful completion of this milestone resulted in a fully functional Flask backend capable of performing carbon emission calculations, handling REST API requests, validating user inputs, and supporting seamless communication with the frontend. This backend serves as the foundation for

integrating Firebase services and AI-powered sustainability recommendations in the subsequent milestones.

Milestone 3: Frontend Development

Milestone 3 focuses on the design and development of the user interface for the EcoTrack application. The frontend was developed using **React.js** with **Vite**, providing a fast, responsive, and interactive user experience. The objective of this milestone was to create a modern web interface that enables users to record daily activities, monitor carbon emissions, visualize environmental impact, and receive AI-powered sustainability recommendations.

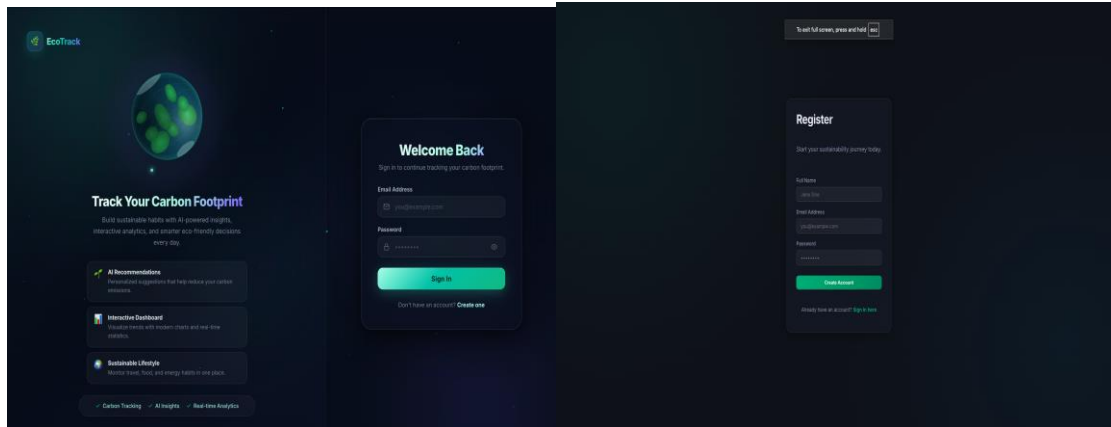
The frontend communicates with the Flask backend through REST APIs and integrates seamlessly with Firebase Authentication for secure user login and Cloud Firestore for data management. Special attention was given to responsive design, user-friendly navigation, and attractive data visualization using interactive charts.

Activity 3.1: User Authentication Module

The first step in frontend development was implementing a secure authentication system. Firebase Authentication was integrated to provide user registration and login functionality. Protected routes were implemented to ensure that only authenticated users could access the dashboard and other application modules.

The following activities were completed:

- Design Login and Registration pages.
- Integrate Firebase Authentication.
- Implement secure login and logout functionality.
- Validate user credentials.
- Redirect authenticated users to the Dashboard.
- Protect application routes from unauthorized access.



Activity 3.2: Dashboard Development

The Dashboard serves as the main interface of EcoTrack, providing users with an overview of their carbon footprint and sustainability progress. The dashboard displays total emissions, Eco Score, category-wise emissions, historical trends, and AI-powered recommendations using interactive charts.

The following features were implemented:

- Display total carbon emissions.
- Display Eco Score.
- Show carbon budget progress.
- Display category-wise emission charts.
- Display recent activity summary.
- Display AI sustainability recommendations.
- Responsive dashboard layout.

Activity 3.4: History Module Development

The History module provides users with access to previously recorded carbon footprint entries. Users can review historical records and analyze changes in their environmental impact over time.

The module includes:

- Display previous activity logs.
- Chronological record listing.
- Carbon emission history.
- User-specific records.
- Integration with Cloud Firestore.

Activity 3.5: AI Insights Module

The AI Insights module provides personalized sustainability recommendations generated using the Groq AI model. Recommendations are based on the user's recorded activities and calculated carbon emissions.

The module performs the following tasks:

- Retrieve user emission data.
- Generate AI-powered recommendations.
- Display practical eco-friendly suggestions.
- Encourage sustainable lifestyle improvements.

Activity 3.6: Profile Module

The Profile module displays user account information and provides an overview of personal sustainability statistics. The page allows users to view their account details and monitor their overall environmental performance.

Features include:

- Display user profile information.
- View account details.
- Display Eco Score.
- Display total emissions.
- Secure logout.

Activity 3.7: Responsive User Interface

To ensure accessibility across multiple devices, the frontend was designed using responsive web design principles. The user interface automatically adapts to different screen sizes, providing a consistent experience on desktops, tablets, and mobile devices.

The following UI improvements were implemented:

- Responsive layouts.
- Glassmorphism design.
- Smooth animations.
- Interactive buttons.
- Responsive navigation.
- Mobile-friendly components.

End of Milestone 3

The successful completion of this milestone resulted in a fully functional and responsive frontend application capable of securely authenticating users, recording daily carbon footprint activities, displaying interactive dashboards, managing historical records, and presenting AI-powered sustainability recommendations. The frontend provides an intuitive and engaging user experience while seamlessly communicating with the backend services.

Milestone 4: Firebase Integration

Milestone 4 focuses on integrating **Firebase Authentication** and **Cloud Firestore** into the EcoTrack application. Firebase provides a secure and scalable cloud platform for managing user authentication and storing application data. The integration ensures that every user's carbon footprint records remain private, synchronized, and accessible across multiple devices.

Firebase Authentication was implemented to provide secure user registration, login, and session management, while Cloud Firestore was used as the primary NoSQL cloud database to store user profiles, daily habit logs, and carbon emission records. This integration enables real-time data synchronization and enhances the overall reliability and scalability of the application.

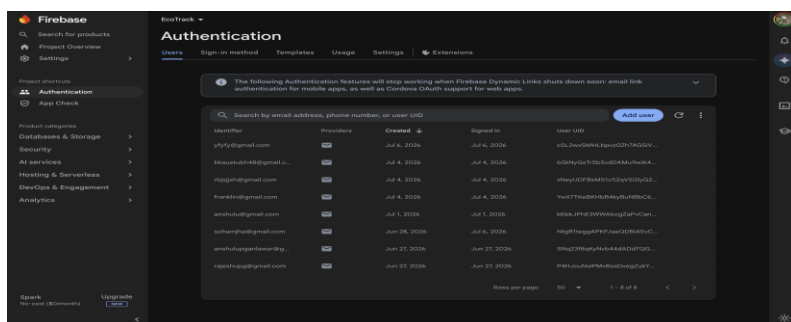
Activity 4.1: Firebase Authentication Integration

The first step of this milestone was integrating Firebase Authentication into the application to provide secure user authentication and account management.

The following activities were completed:

- Create a Firebase project.
- Configure Firebase Authentication.
- Enable Email and Password authentication.
- Integrate Firebase SDK into the React application.
- Implement user registration functionality.
- Implement secure user login.
- Maintain authenticated user sessions.
- Protect application routes from unauthorized access.
- Implement secure logout functionality.

The authentication system ensures that each user can securely access only their own data while preventing unauthorized access to the application.



Activity 4.2: Cloud Firestore Database Integration

Cloud Firestore was integrated as the primary database to store user information and carbon footprint records.

The following activities were performed:

- Create Cloud Firestore database.
- Configure Firestore security rules.
- Create user data collections.
- Store daily habit logs.
- Retrieve user-specific carbon footprint records.
- Display historical records on the History page.
- Synchronize data across multiple devices.
- Ensure secure read and write operations.

Firestore enables efficient storage and retrieval of structured data while providing scalability, reliability, and real-time synchronization.

Activity 4.3: User Data Management

User-specific data management was implemented to ensure that each authenticated user can securely access and manage their own carbon footprint information.

The following functionalities were implemented:

- Store user profile information.
- Save transportation, food, and energy activity logs.
- Retrieve dashboard statistics.
- Display historical emission records.
- Manage user sessions.
- Prevent unauthorized access to other users' data.

This approach ensures data privacy and improves the overall security of the EcoTrack application.

Firebase Database Structure

The Cloud Firestore database consists of separate collections that organize application data efficiently.

Users Collection

Stores authenticated user information including:

- User ID
- Name
- Email Address
- Registration Date
- Eco Score

Habit Logs Collection

Stores daily activity records such as:

- Transportation Details
- Food Consumption
- Energy Usage

- Carbon Emissions
- Timestamp

Database Features

The implemented Firestore database provides:

- Cloud-based storage
- Real-time synchronization
- Secure authentication
- Scalable NoSQL architecture
- Fast data retrieval
- User-specific document access

End of Milestone 4

The successful integration of Firebase Authentication and Cloud Firestore established a secure cloud infrastructure for the EcoTrack application. Authentication ensures secure access to user accounts, while Firestore provides reliable cloud storage for user profiles and carbon footprint records. This milestone significantly improved the application's security, scalability, and data management capabilities.

Milestone 5: Deployment

Milestone 5 focuses on deploying the EcoTrack application to cloud hosting platforms, making it accessible over the internet. The deployment process involved preparing both the frontend and backend applications for production, configuring environment variables, and hosting the services using **Render** and **Vercel**. Version control was managed using **GitHub**, enabling continuous deployment whenever new code was pushed to the repository.

The backend was deployed using **Render**, while the frontend was hosted on **Vercel**. Both platforms were connected to the GitHub repository, allowing automatic builds and deployments after each successful commit.

Activity 5.1: Backend Deployment on Render

The Flask backend was deployed using **Render**, a cloud platform that supports Python web applications. Before deployment, the backend project was configured with the necessary dependencies, startup commands, and environment variables.

The following activities were performed:

- Create a Render account.
- Connect the GitHub repository.
- Create a new Web Service.
- Select the backend folder as the root directory.

- Configure the build command.
- Configure the start command.
- Add environment variables.
- Deploy the Flask application.
- Verify API endpoints after deployment.

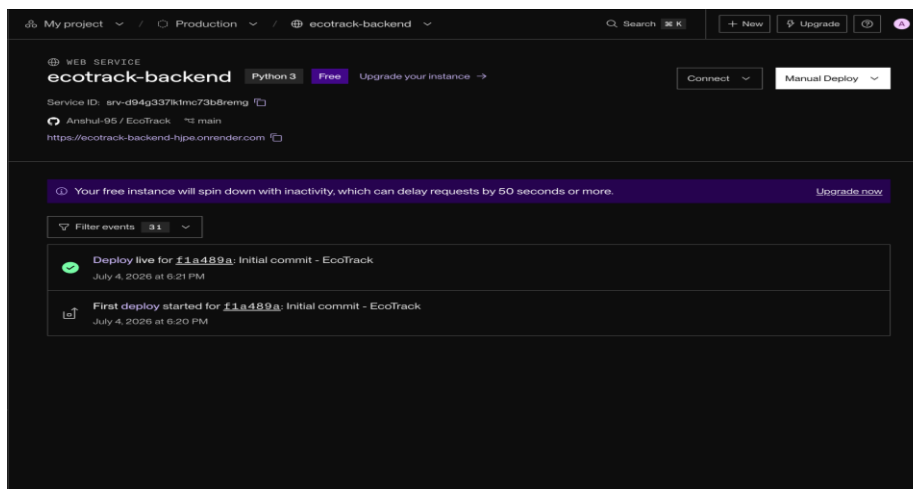
Build Command:

```
pip install -r requirements.txt
```

Start Command

```
python app.py
```

After deployment, the backend API was successfully hosted on Render and became publicly accessible through a secure HTTPS endpoint.



Activity 5.2: Frontend Deployment on Vercel

The React frontend was deployed using **Vercel**, a cloud platform optimized for modern frontend applications. Vercel automatically builds and deploys the application whenever new changes are pushed to GitHub.

The following deployment activities were completed:

- Create a Vercel account.
- Import the GitHub repository.
- Select the frontend folder as the project root.
- Configure Vite as the build framework.
- Add Firebase environment variables.
- Configure the backend API URL.
- Deploy the application.
- Verify successful deployment.

The frontend was successfully deployed and connected to the Flask backend hosted on Render.

(Insert Vercel Deployment Screenshot Here)

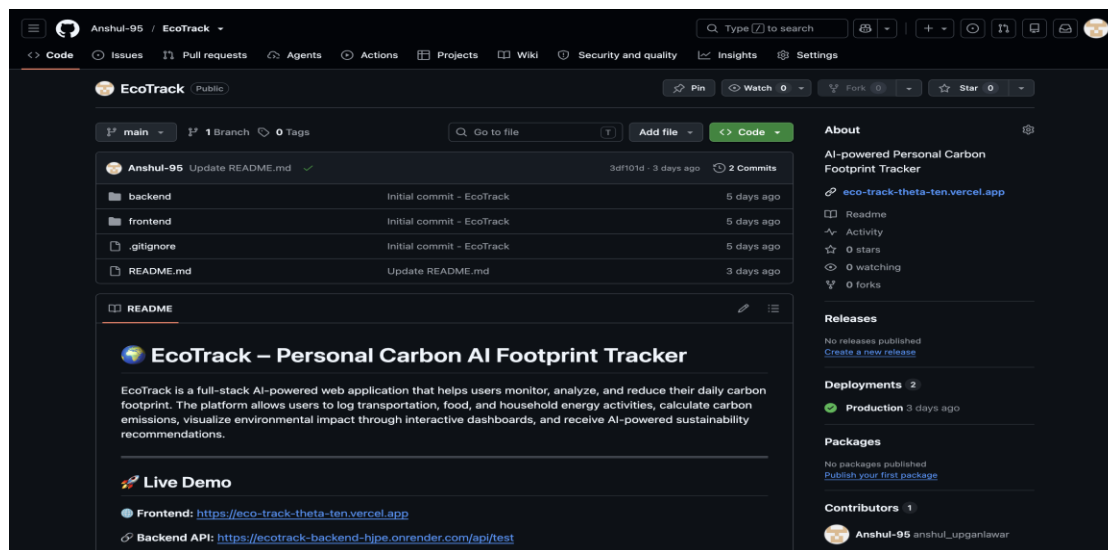
Activity 5.3: GitHub Repository Integration

GitHub was used as the version control platform throughout the development of EcoTrack. It enabled source code management, collaboration, and seamless integration with both Render and Vercel for automatic deployments.

The following tasks were performed:

- Initialize the Git repository.
- Configure the remote GitHub repository.
- Push frontend and backend source code.
- Manage project versions using Git.
- Enable automatic deployment through GitHub integration.

This setup ensured that every update pushed to the repository automatically triggered a new deployment, keeping the production application synchronized with the latest source code.



Activity 5.4: Deployment Verification

After deploying both the frontend and backend, the complete application was tested to ensure all modules functioned correctly in the production environment.

The following functionalities were verified:

- User Registration
- User Login
- Dashboard Loading
- Habit Logging
- Carbon Emission Calculation

- History Management
- AI Insights
- Profile Management
- Firebase Authentication
- Cloud Firestore Connectivity
- Backend API Communication

Successful testing confirmed that the deployed application was fully operational and accessible from any device with an internet connection.

Live Application

Frontend (Vercel):

<https://eco-track-theta-ten.vercel.app>

Backend (Render):

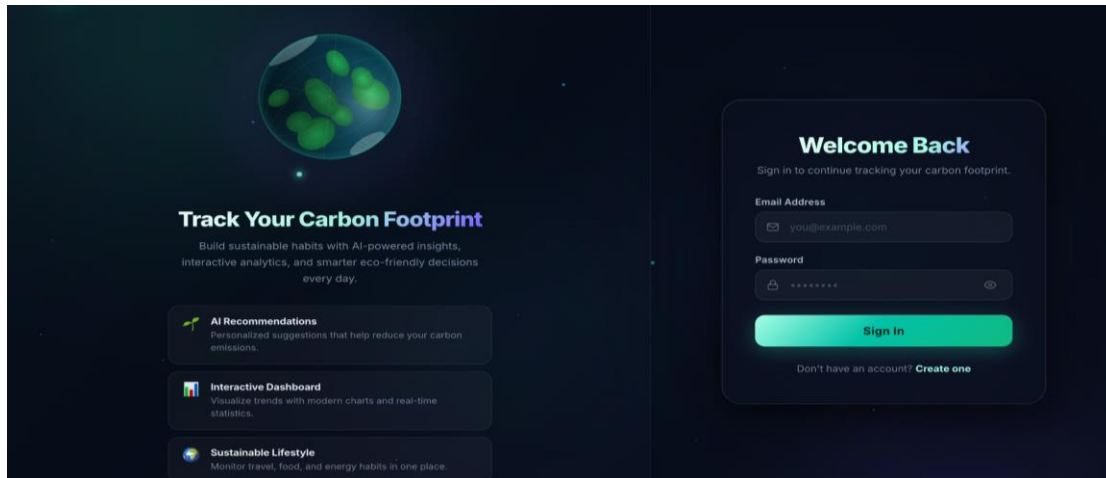
<https://ecotrack-backend-hjpe.onrender.com/api/test>

End of Milestone 5

The successful deployment of EcoTrack marked the completion of the development phase. By hosting the frontend on Vercel and the backend on Render, the application became publicly accessible, scalable, and reliable. Integration with GitHub enabled continuous deployment, ensuring that future updates could be automatically published with minimal effort. The deployment process demonstrated the practical implementation of modern cloud hosting technologies and completed the full-stack development lifecycle of the EcoTrack application.

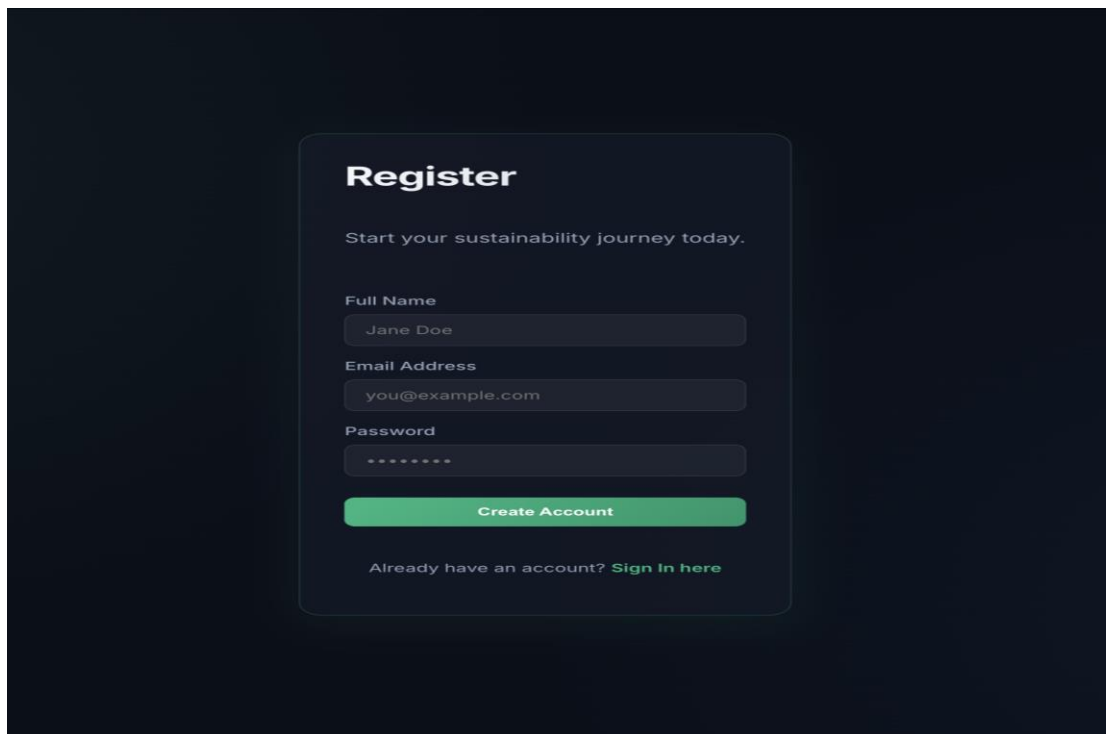
Exploring the Website's Web Pages:

Login Page:



Description: The Login page serves as the entry point of the EcoTrack application. It allows registered users to securely authenticate using Firebase Authentication. The interface is designed with a modern glassmorphism theme, providing a simple and intuitive experience. After successful authentication, users are redirected to the Dashboard.

Registration Page:



Description: The Registration page enables new users to create an EcoTrack account. User credentials are securely stored using Firebase Authentication. After successful registration, users can immediately access the application and begin tracking their carbon footprint.

Dashboard:



Description: The Dashboard is the central module of EcoTrack. It provides users with a comprehensive overview of their environmental impact through interactive charts, carbon emission statistics, Eco Score, and AI-powered sustainability recommendations. The dashboard enables users to monitor their daily progress and visualize their carbon footprint effectively.

Habit Logger:

The EcoTrack Habit Logger interface is designed for users to log their daily habits to estimate their carbon footprint. It features a sidebar with navigation options: Dashboard, Log Habits, History, AI Insights, and Profile. The main content area includes a greeting, a daily carbon entry section, and a progress bar indicating the user's progress through four steps: Travel, Food, Energy, and Summary.

The **Travel** step is currently active, showing a list of transportation options:

- Car Petrol:** Private commute
- Car Diesel:** Private commute
- Bike:** Motorbike / Scooter
- Bus:** Public transit
- Train:** Rail / Metro
- Flight:** Aviation
- Walking:** Zero direct CO₂
- Cycling:** Zero direct CO₂

The **LIVE SUMMARY** section shows the following data:

- Travel:** 0 kg
- Food:** 2.5 kg
- Energy:** 0 kg

The **ESTIMATED CARBON FOOTPRINT** is 2.5 kg CO₂.

The user's profile information is displayed at the bottom left: Soham Jha, sohamjha@gmail.com, with a Logout button.

Description:

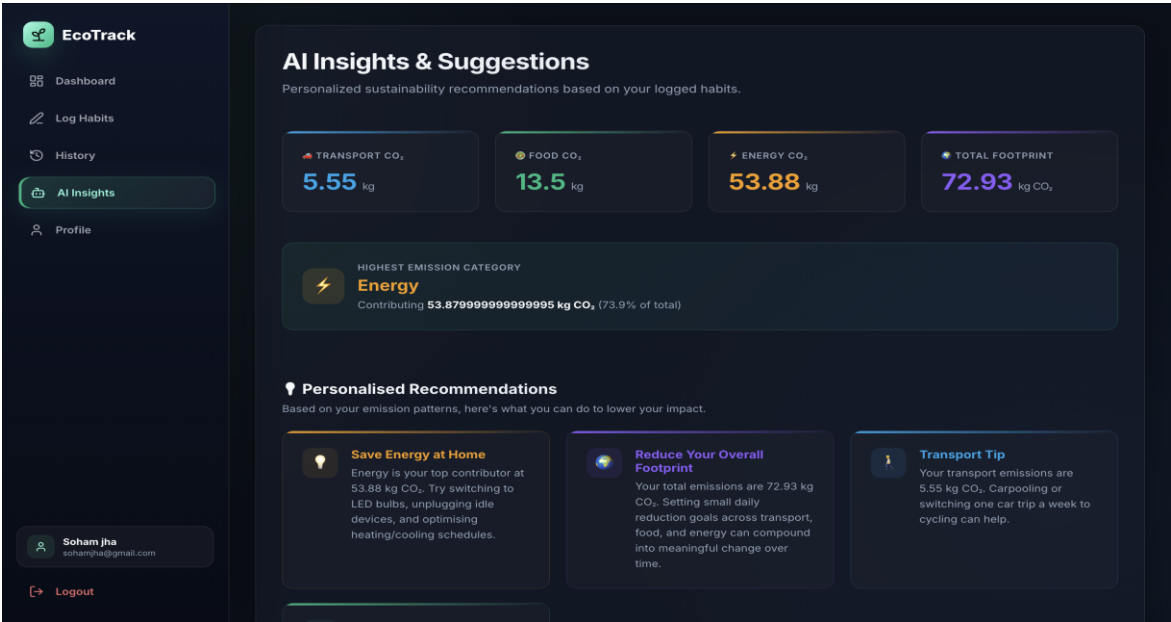
The Habit Logger enables users to record daily activities related to transportation, food consumption, and household energy usage. Based on the entered information, the system calculates the corresponding carbon emissions and stores the records securely in Cloud Firestore.

History:



Description: The History page displays previously recorded carbon footprint data in chronological order. Users can review their past activities and analyze emission trends over time.

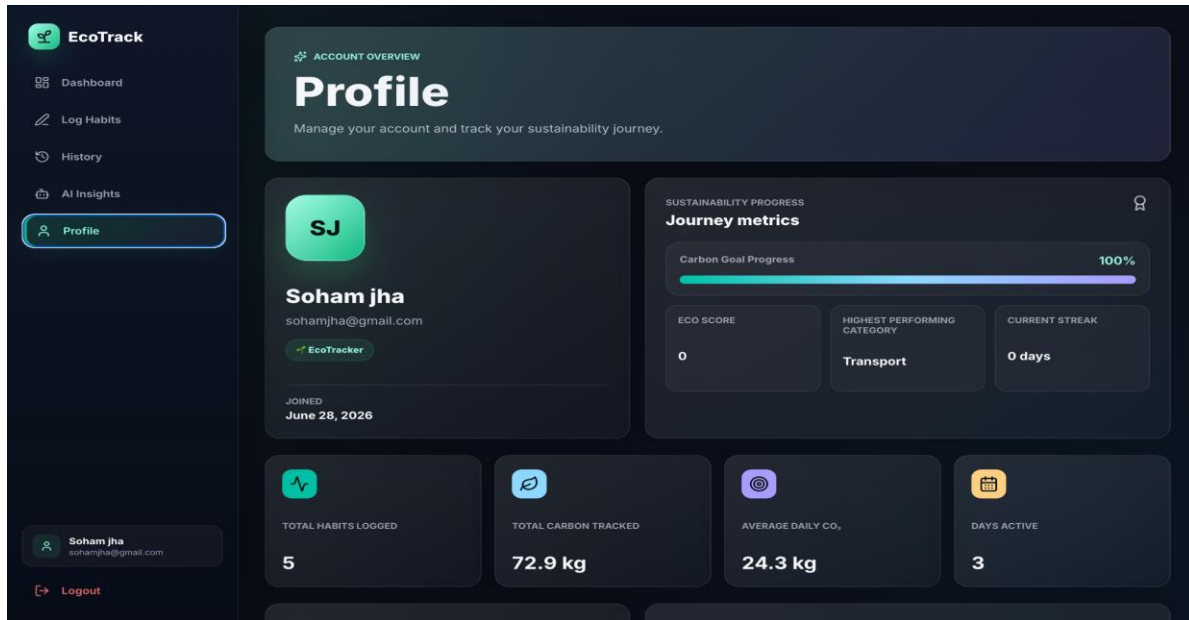
AI Insights:



Description:

The AI Insights page provides personalized sustainability recommendations generated using Groq's LLaMA 3.3-70B Versatile model. The suggestions are based on the user's recorded activities and aim to promote environmentally friendly habits.

Profile:



Description:

The Profile page displays the authenticated user's account information and sustainability statistics. It provides quick access to personal details and overall environmental performance.

Conclusion

The successful completion of the EcoTrack project demonstrates the practical implementation of a full-stack web application that combines cloud computing, artificial intelligence, and environmental sustainability. Throughout the project, modern web technologies were integrated to develop a responsive, secure, and scalable platform for monitoring personal carbon emissions.

The application enables users to securely record their daily transportation, food consumption, and household energy usage while automatically calculating their carbon footprint using predefined emission factors. By integrating **Firestore Authentication** and **Cloud Firestore**, EcoTrack provides secure user management and reliable cloud-based data storage. The integration of **Groq's LLaMA 3.3-70B Versatile** model enhances the application by generating personalized sustainability recommendations that encourage environmentally responsible behavior.

The frontend, developed using **React.js** and **Vite**, provides an interactive and responsive user experience, while the **Flask** backend efficiently handles API requests, business logic, and carbon emission calculations. The successful deployment of the application using **Vercel** and **Render**

demonstrates the effectiveness of modern cloud-hosting platforms in delivering scalable web applications.

Overall, EcoTrack serves as a practical solution for increasing environmental awareness by helping users understand their carbon footprint and encouraging sustainable lifestyle choices through intelligent insights and data visualization.